

Средства симуляции ЦП и ОС

Лекция №4

Державин Андрей
Шурыгин Антон

Виртуальная память

Управление памятью в современных ОС

- Одна из функций ОС – эффективное управление памятью
- Память – ограниченный ресурс
- На современных ОС общего назначения существует запрос на многозадачность

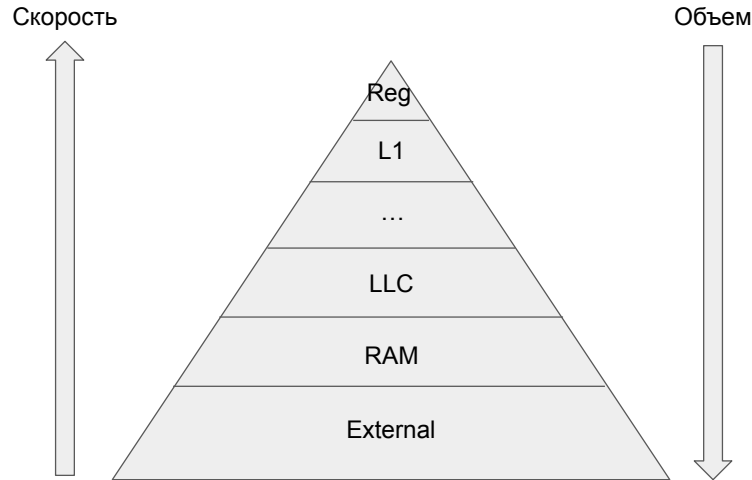


Рис. 1. Иерархия компьютерной памяти

Проблематика отображения адресного пространства

- Виртуальная память
 - Механизм защиты адресного пространства
 - Распределение памяти между процессами
 - Оптимальность
- Реализация виртуальной памяти – совместная работа аппаратной и программной частей

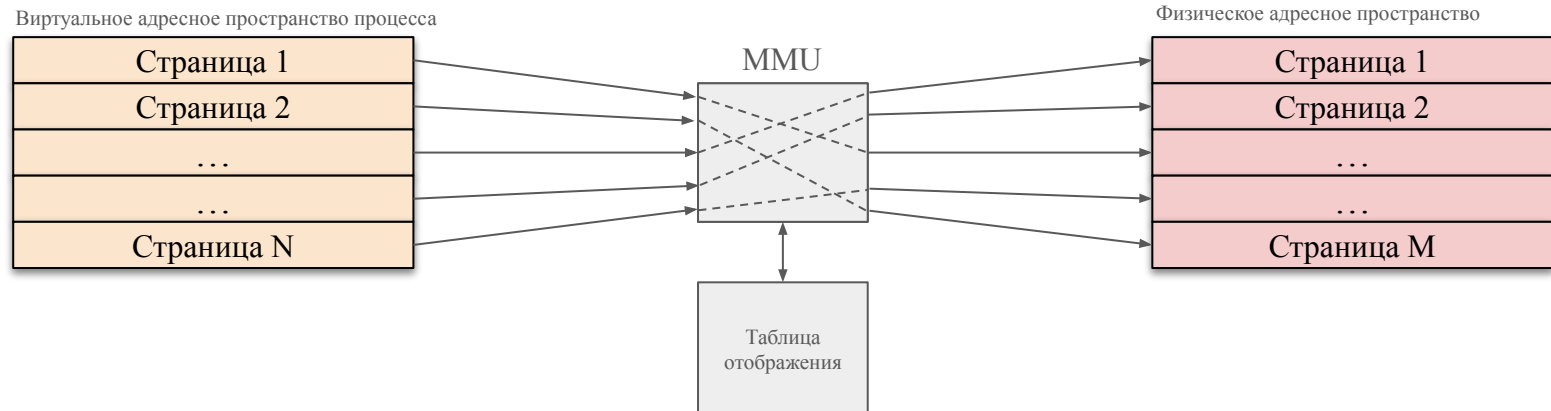


Рис. 2. Связь виртуальной и физической памяти

Организация отображения

- Сегментная

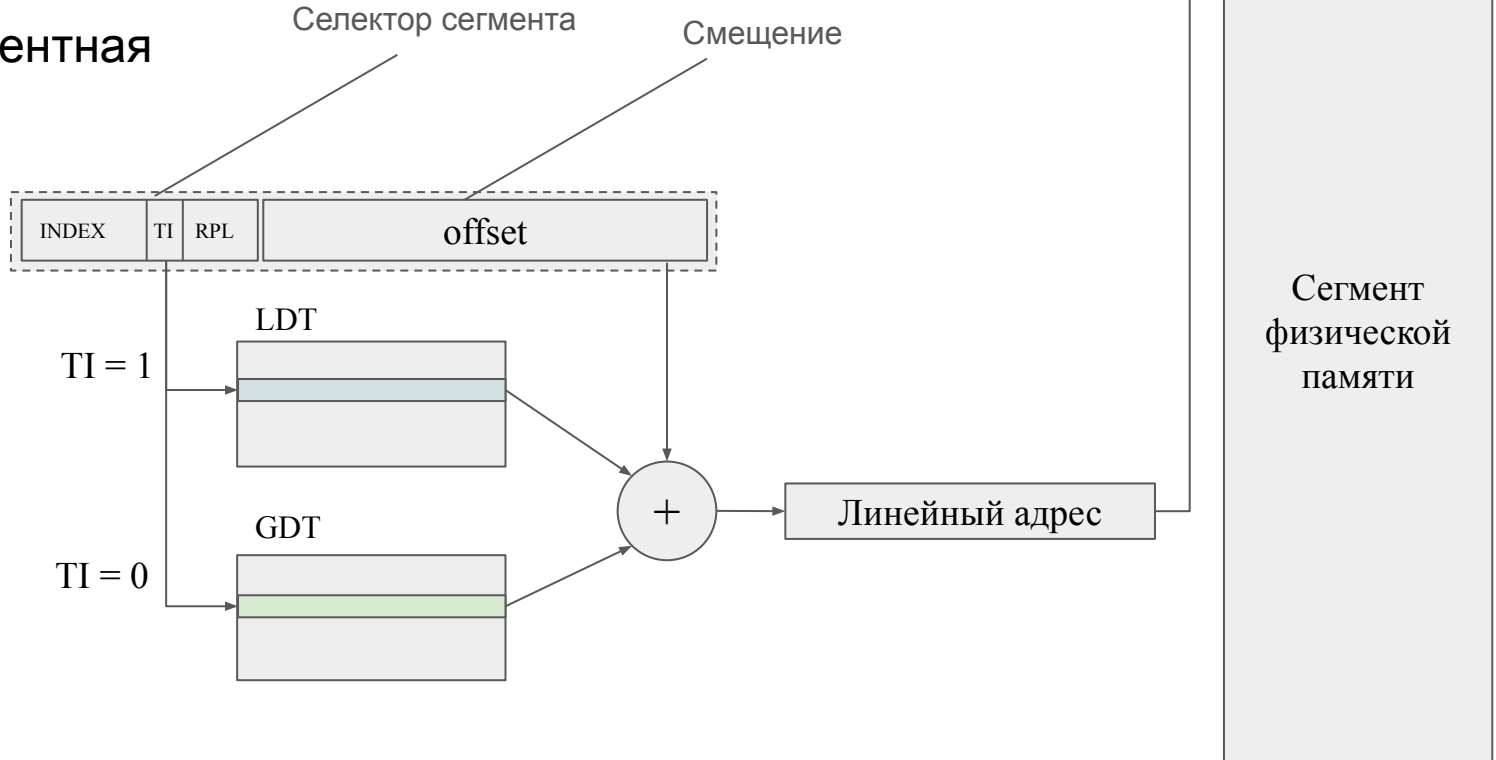


Рис.3. Трансляция адреса с использованием механизма сегментации

Организация отображения

- Страничная, сегментно-страничная

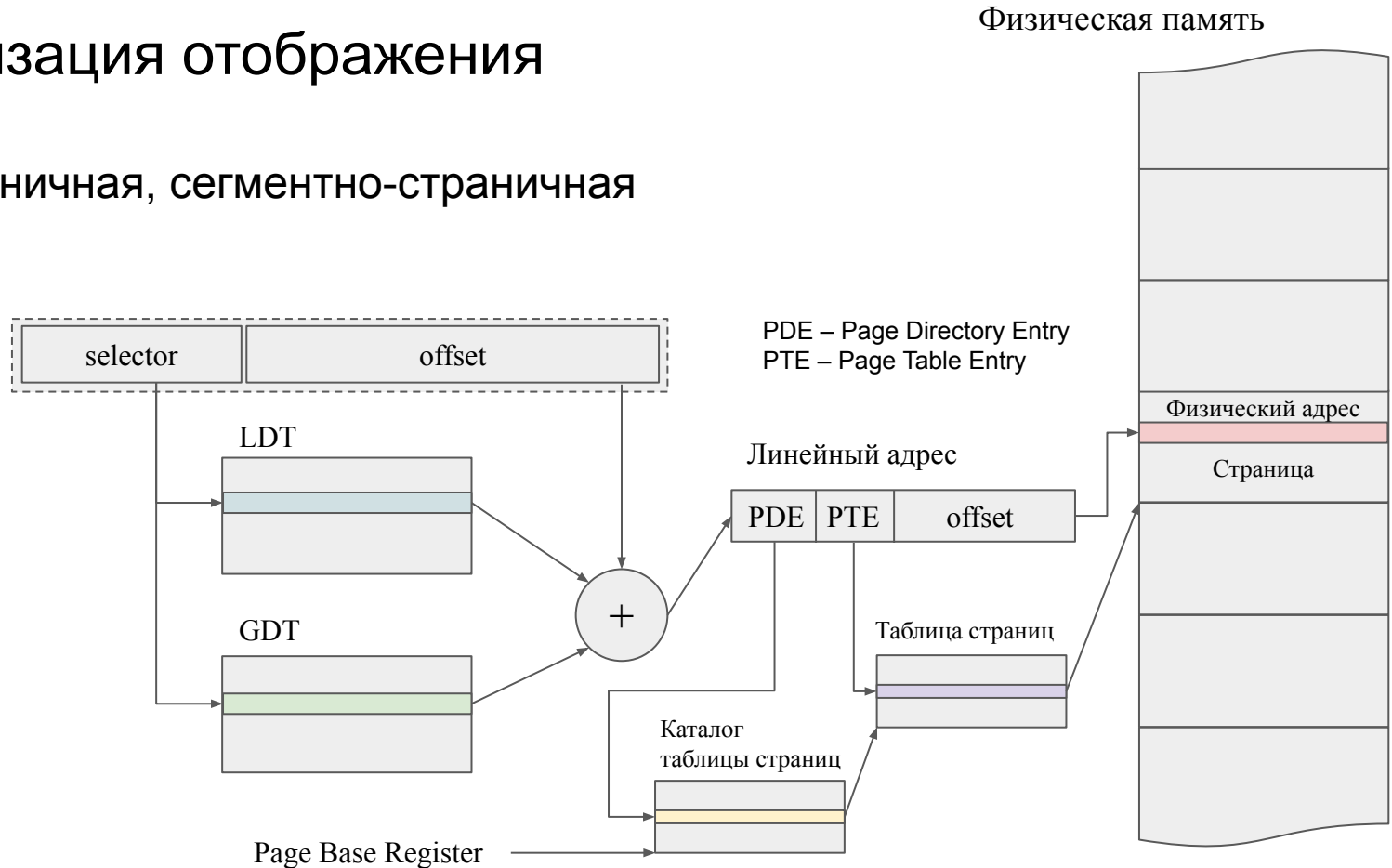
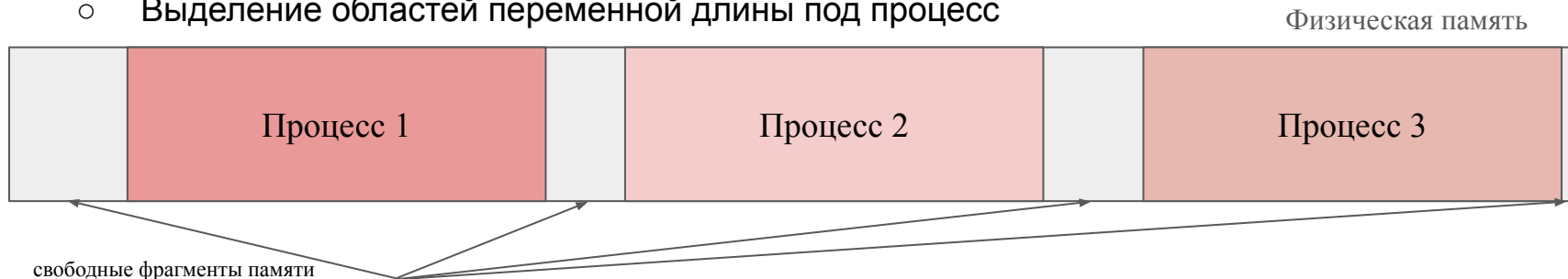


Рис.4. Трансляция адреса с использованием страничного-сегментного механизма

Фрагментация памяти

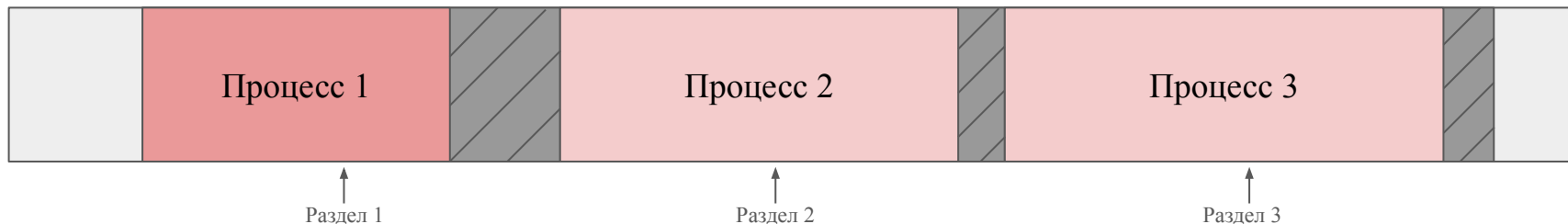
- Внешняя фрагментация

- Выделение областей переменной длины под процесс



- Внутренняя фрагментация

- Память выделяется, но не используется процессом полностью



Всё круто, но...

Как программа попадает в память?



Формат ELF

Формат ELF

- ELF (**E**xecutable and **L**inkable **F**ormat) - формат бинарных файлов
- В ELF можно хранить:
 - Исполняемые файлы (executable file)
 - Объектные файлы (relocatable file)
 - Разделяемые библиотеки (shared object)
- Два представления ELF-файла:
 - Секции - для разработки (ассемблер, компоновщик, отладчик)
 - .text, .data, .rodata...
 - Сегменты - для ОС
 - Позволяет оптимизировать время загрузки в ОС - загружаем только сегменты помеченные флагом PT_LOAD

Формат ELF

- ELF файл состоит из:
 - Заголовок
 - Содержимое
 - Таблица заголовков сегментов (“Program header”)
 - Таблица заголовков секций (“Section header”)
 - Данные

Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```

“ELF”

0x7F 0x45 0x4C 0x46

Разрядность: 32(1) или 64(2)

Порядок байт: little-(1) или big-(2) endian

Версия формата

Используемое ABI (0x0 - SystemV, 0x3 - Linux,...)

Версия ABI

Паддинг (нули)

Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```

Тип файла

Значение	Описание
0x01	Relocatable
0x02	Executable
0x03	Shared object
0x04	Core file

Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```

ISA

Значение	Описание
0x08	MIPS
...	...
0x3E	x86_64
...	...
0xF3	RISC-V

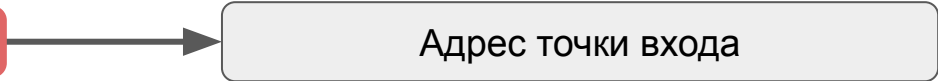
Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```

Версия ELF (опять?)

Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```

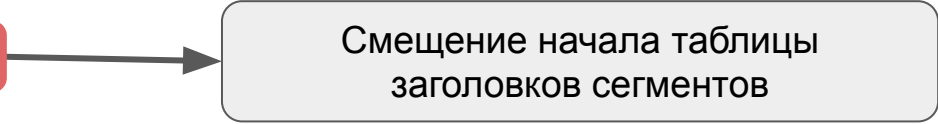


Адрес точки входа

The diagram consists of a red rectangular box highlighting the `Elf64_Addr e_entry;` line in the ELF header structure definition. A black arrow points from this box to a light gray rounded rectangular box on the right containing the text "Адрес точки входа" (Entry point address).

Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```



Смещение начала таблицы
заголовков сегментов

The diagram consists of a red rectangular box highlighting the `e_phoff` field in the ELF header structure. A black arrow points from this box to a grey rounded rectangular box containing the Russian text: "Смещение начала таблицы заголовков сегментов" (Offset of the start of the segment header table).

Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```



Смещение начала таблицы
заголовков секций

Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```

Флаги (зависят от архитектуры)

Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```

Размер заголовка:
64(64) либо 52(32) байт

Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```

Размер заголовка сегмента:
64(64) либо 40(32) байт

Количество заголовков сегментов

Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```

Размер заголовка секции:
40(64) либо 32(32) байт

Количество заголовков секций

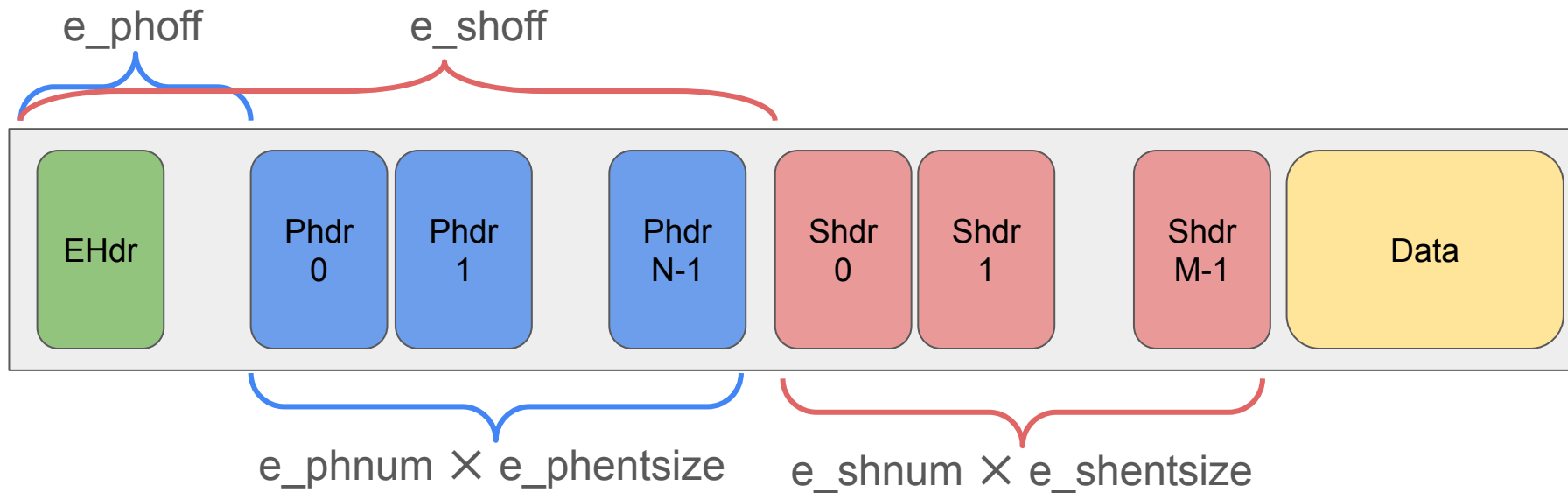
Формат ELF. Заголовок

```
typedef struct elf64_hdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
} Elf64_Ehdr;
```



Номер секции в таблице секций, в которой содержатся имена секций

Формат ELF. Общая структура



Формат ELF. Заголовок сегмента

```
typedef struct elf64_phdr {  
    Elf64_Word p_type;  
    Elf64_Word p_flags;  
    Elf64_Off p_offset;  
    Elf64_Addr p_vaddr;  
    Elf64_Addr p_paddr;  
    Elf64_Xword p_filesz;  
    Elf64_Xword p_memsz;  
    Elf64_Xword p_align;  
} Elf64_Phdr;
```

Тип сегмента

Значение	Описание
...	
0x1	Загружаемый
0x2	Информация для динамической линковки
0x3	Информация для интерпретатора
...	

Формат ELF. Заголовок сегмента

```
typedef struct elf64_phdr {  
    Elf64_Word p_type;  
    Elf64_Word p_flags;  
    Elf64_Off p_offset;  
    Elf64_Addr p_vaddr;  
    Elf64_Addr p_paddr;  
    Elf64_Xword p_filesz;  
    Elf64_Xword p_memsz;  
    Elf64_Xword p_align;  
} Elf64_Phdr;
```

Флаги доступа

Значение	Описание
0x1	Исполнение
0x2	Запись
0x4	Чтение

Формат ELF. Заголовок сегмента

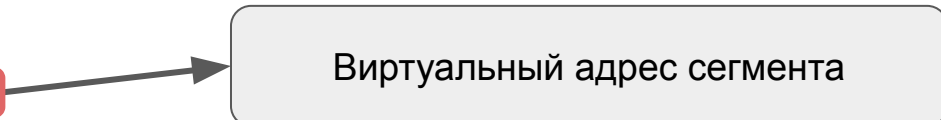
```
typedef struct elf64_phdr {  
    Elf64_Word p_type;  
    Elf64_Word p_flags;  
    Elf64_Off p_offset;  
    Elf64_Addr p_vaddr;  
    Elf64_Addr p_paddr;  
    Elf64_Xword p_filesz;  
    Elf64_Xword p_memsz;  
    Elf64_Xword p_align;  
} Elf64_Phdr;
```



Смещение данных сегмента

Формат ELF. Заголовок сегмента

```
typedef struct elf64_phdr {  
    Elf64_Word p_type;  
    Elf64_Word p_flags;  
    Elf64_Off p_offset;  
    Elf64_Addr p_vaddr;  
    Elf64_Addr p_paddr;  
    Elf64_Xword p_filesz;  
    Elf64_Xword p_memsz;  
    Elf64_Xword p_align;  
} Elf64_Phdr;
```



Виртуальный адрес сегмента

The diagram consists of a red rectangular box highlighting the `Elf64_Addr p_vaddr;` line in the code block on the left. A black arrow points from this box to a light gray rounded rectangular box on the right containing the text "Виртуальный адрес сегмента".

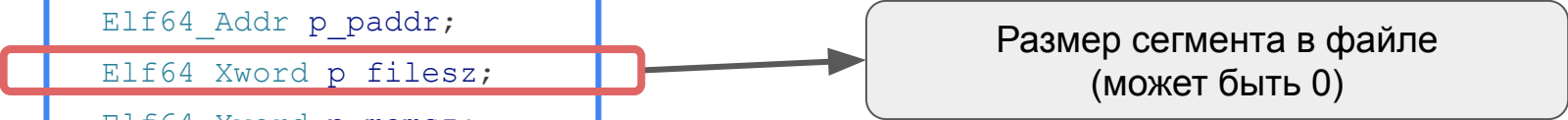
Формат ELF. Заголовок сегмента

```
typedef struct elf64_phdr {  
    Elf64_Word p_type;  
    Elf64_Word p_flags;  
    Elf64_Off p_offset;  
    Elf64_Addr p_vaddr;  
    Elf64_Addr p_paddr;  
    Elf64_Xword p_filesz;  
    Elf64_Xword p_memsz;  
    Elf64_Xword p_align;  
} Elf64_Phdr;
```

Физический адрес сегмента
(обычно не используется)

Формат ELF. Заголовок сегмента

```
typedef struct elf64_phdr {  
    Elf64_Word p_type;  
    Elf64_Word p_flags;  
    Elf64_Off p_offset;  
    Elf64_Addr p_vaddr;  
    Elf64_Addr p_paddr;  
    Elf64_Xword p_filesz;  
    Elf64_Xword p_memsz;  
    Elf64_Xword p_align;  
} Elf64_Phdr;
```



Размер сегмента в файле
(может быть 0)

Формат ELF. Заголовок сегмента

```
typedef struct elf64_phdr {  
    Elf64_Word p_type;  
    Elf64_Word p_flags;  
    Elf64_Off p_offset;  
    Elf64_Addr p_vaddr;  
    Elf64_Addr p_paddr;  
    Elf64_Xword p_filesz;  
    Elf64_Xword p_memsz;  
    Elf64_Xword p_align;  
} Elf64_Phdr;
```

Размер сегмента в памяти
(может быть 0)

Формат ELF. Заголовок сегмента

```
typedef struct elf64_phdr {  
    Elf64_Word p_type;  
    Elf64_Word p_flags;  
    Elf64_Off p_offset;  
    Elf64_Addr p_vaddr;  
    Elf64_Addr p_paddr;  
    Elf64_Xword p_filesz;  
    Elf64_Xword p_memsz;  
    Elf64_Xword p_align;  
} Elf64_Phdr;
```

Выравнивание

Формат ELF. Заголовок секции

```
typedef struct elf64_shdr {  
    Elf64_Word sh_name;  
    Elf64_Word sh_type;  
    Elf64_Xword sh_flags;  
    Elf64_Addr sh_addr;  
    Elf64_Off sh_offset;  
    Elf64_Xword sh_size;  
    Elf64_Word sh_link;  
    Elf64_Word sh_info;  
    Elf64_Xword sh_addralign;  
    Elf64_Xword sh_entsize;  
} Elf64_Shdr;
```

Смещение имени секции в .shstrtab

Формат ELF. Заголовок секции

```
typedef struct elf64_shdr {  
    Elf64_Word sh_name;  
    Elf64_Word sh_type;  
    Elf64_Xword sh_flags;  
    Elf64_Addr sh_addr;  
    Elf64_Off sh_offset;  
    Elf64_Xword sh_size;  
    Elf64_Word sh_link;  
    Elf64_Word sh_info;  
    Elf64_Xword sh_addralign;  
    Elf64_Xword sh_entsize;  
} Elf64_Shdr;
```

Тип секции (данные, строки, таблица символов...)

Формат ELF. Заголовок секции

```
typedef struct elf64_shdr {  
    Elf64_Word sh_name;  
    Elf64_Word sh_type;  
    Elf64_Xword sh_flags;  
    Elf64_Addr sh_addr;  
    Elf64_Off sh_offset;  
    Elf64_Xword sh_size;  
    Elf64_Word sh_link;  
    Elf64_Word sh_info;  
    Elf64_Xword sh_addralign;  
    Elf64_Xword sh_entsize;  
} Elf64_Shdr;
```

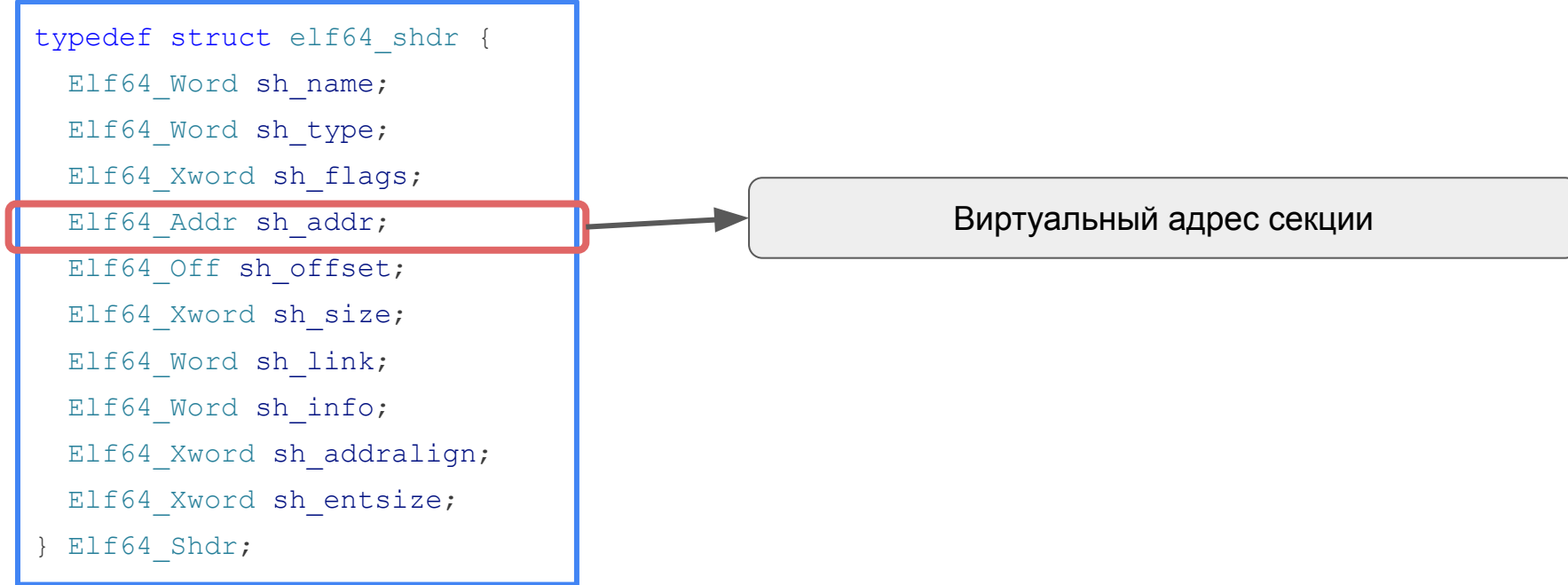


Флаги

The diagram consists of a light gray rounded rectangle with the word 'Флаги' (Flags) inside. A black arrow points from the 'sh_flags' field in the code block to this rectangle.

Формат ELF. Заголовок секции

```
typedef struct elf64_shdr {  
    Elf64_Word sh_name;  
    Elf64_Word sh_type;  
    Elf64_Xword sh_flags;  
    Elf64_Addr sh_addr;  
    Elf64_Off sh_offset;  
    Elf64_Xword sh_size;  
    Elf64_Word sh_link;  
    Elf64_Word sh_info;  
    Elf64_Xword sh_addralign;  
    Elf64_Xword sh_entsize;  
} Elf64_Shdr;
```



Виртуальный адрес секции

The diagram illustrates the relationship between a field in the ELF section header and its meaning. On the left, a blue-bordered box contains the C code for the `elf64_shdr` structure. The line `Elf64_Addr sh_addr;` is highlighted with a red rectangle. A black arrow points from this red rectangle to a light gray rounded rectangle on the right, which contains the text "Виртуальный адрес секции" (Virtual address of the section).

Формат ELF. Заголовок секции

```
typedef struct elf64_shdr {  
    Elf64_Word sh_name;  
    Elf64_Word sh_type;  
    Elf64_Xword sh_flags;  
    Elf64_Addr sh_addr;  
    Elf64_Off sh_offset;  
    Elf64_Xword sh_size;  
    Elf64_Word sh_link;  
    Elf64_Word sh_info;  
    Elf64_Xword sh_addralign;  
    Elf64_Xword sh_entsize;  
} Elf64_Shdr;
```

Смещение данных секции

Формат ELF. Заголовок секции

```
typedef struct elf64_shdr {  
    Elf64_Word sh_name;  
    Elf64_Word sh_type;  
    Elf64_Xword sh_flags;  
    Elf64_Addr sh_addr;  
    Elf64_Off sh_offset;  
    Elf64_Xword sh_size;  
    Elf64_Word sh_link;  
    Elf64_Word sh_info;  
    Elf64_Xword sh_addralign;  
    Elf64_Xword sh_entsize;  
} Elf64_Shdr;
```

Размер секции (может быть 0)

Формат ELF. Заголовок секции

```
typedef struct elf64_shdr {  
    Elf64_Word sh_name;  
    Elf64_Word sh_type;  
    Elf64_Xword sh_flags;  
    Elf64_Addr sh_addr;  
    Elf64_Off sh_offset;  
    Elf64_Xword sh_size;  
    Elf64_Word sh_link;  
    Elf64_Word sh_info;  
    Elf64_Xword sh_addralign;  
    Elf64_Xword sh_entsize;  
} Elf64_Shdr;
```

Оptionальные поля
(индекс связанной секции и дополнительная информация)

Формат ELF. Заголовок секции

```
typedef struct elf64_shdr {  
    Elf64_Word sh_name;  
    Elf64_Word sh_type;  
    Elf64_Xword sh_flags;  
    Elf64_Addr sh_addr;  
    Elf64_Off sh_offset;  
    Elf64_Xword sh_size;  
    Elf64_Word sh_link;  
    Elf64_Word sh_info;  
    Elf64_Xword sh_addralign;  
    Elf64_Xword sh_entsize;  
} Elf64_Shdr;
```

Выравнивание

Формат ELF. Заголовок секции

```
typedef struct elf64_shdr {  
    Elf64_Word sh_name;  
    Elf64_Word sh_type;  
    Elf64_Xword sh_flags;  
    Elf64_Addr sh_addr;  
    Elf64_Off sh_offset;  
    Elf64_Xword sh_size;  
    Elf64_Word sh_link;  
    Elf64_Word sh_info;  
    Elf64_Xword sh_addralign;  
    Elf64_Xword sh_entsize;  
} Elf64_Shdr;
```

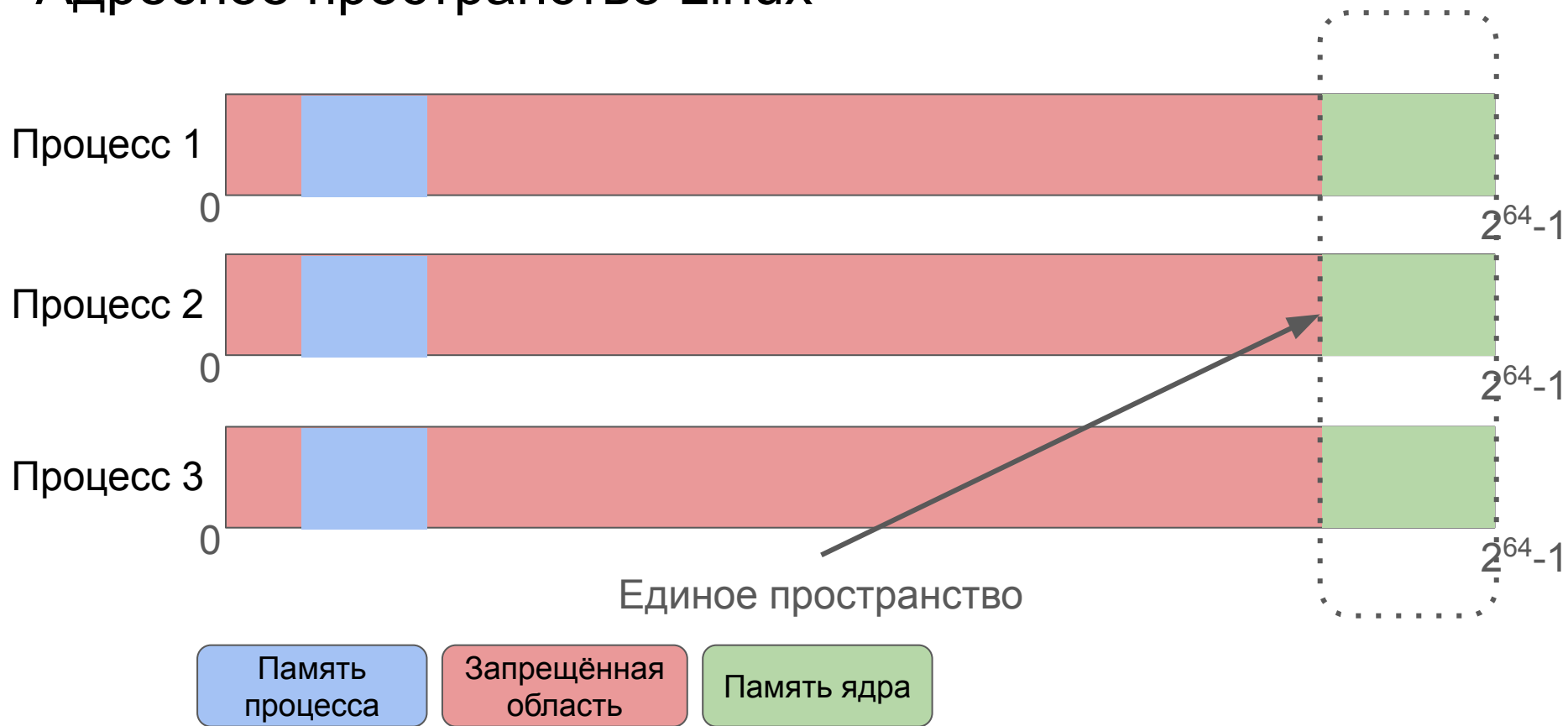
Размер элемента таблицы (опционально).
Напр. нужен для таблицы символов

Адресное пространство Linux

Адресное пространство Linux

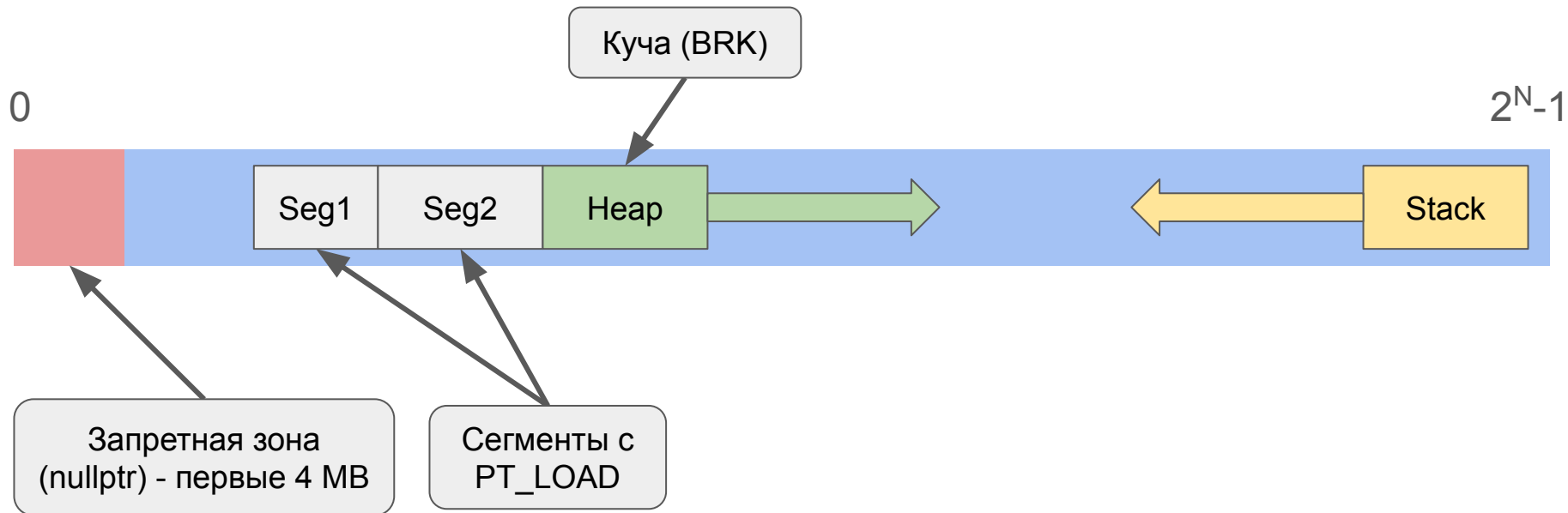
- Как устроена память процесса в 64-битном Linux?

Адресное пространство Linux



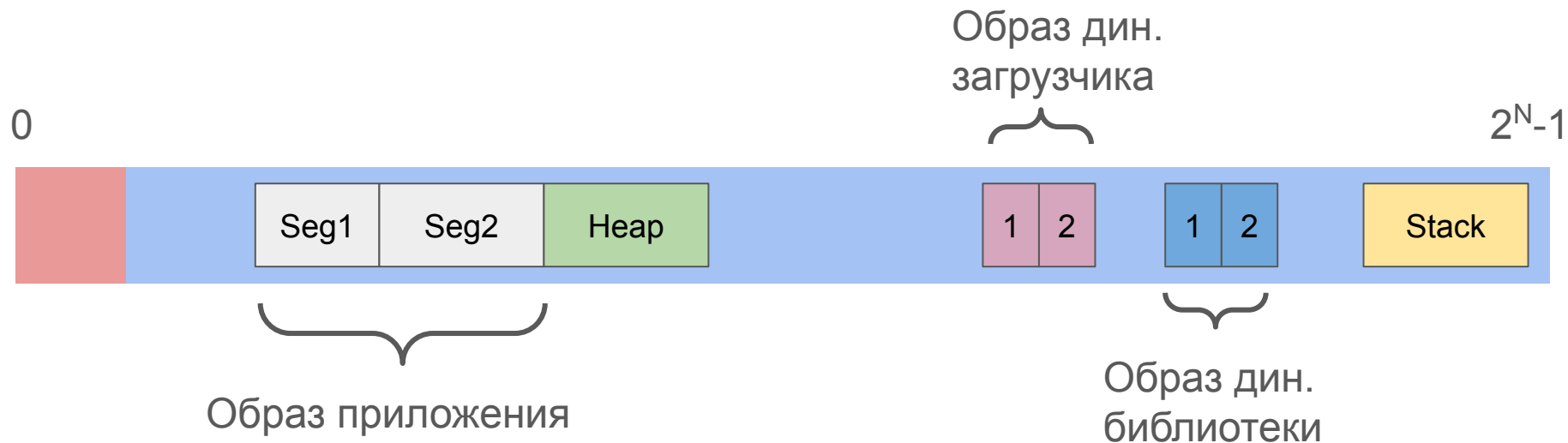
Адресное пространство Linux

- Рассмотрим память процесса



Адресное пространство Linux

➤ Динамическая загрузка

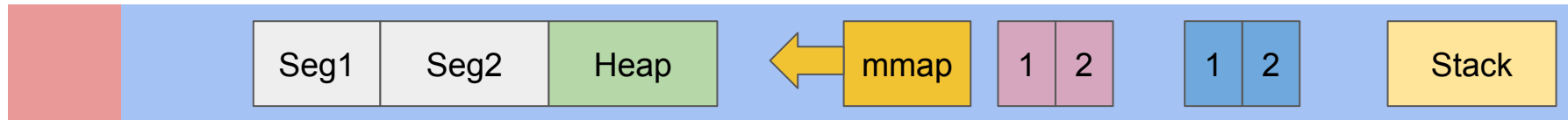


Адресное пространство Linux

- Системный вызов `mmap()`
 - Выделение/освобождение памяти

0

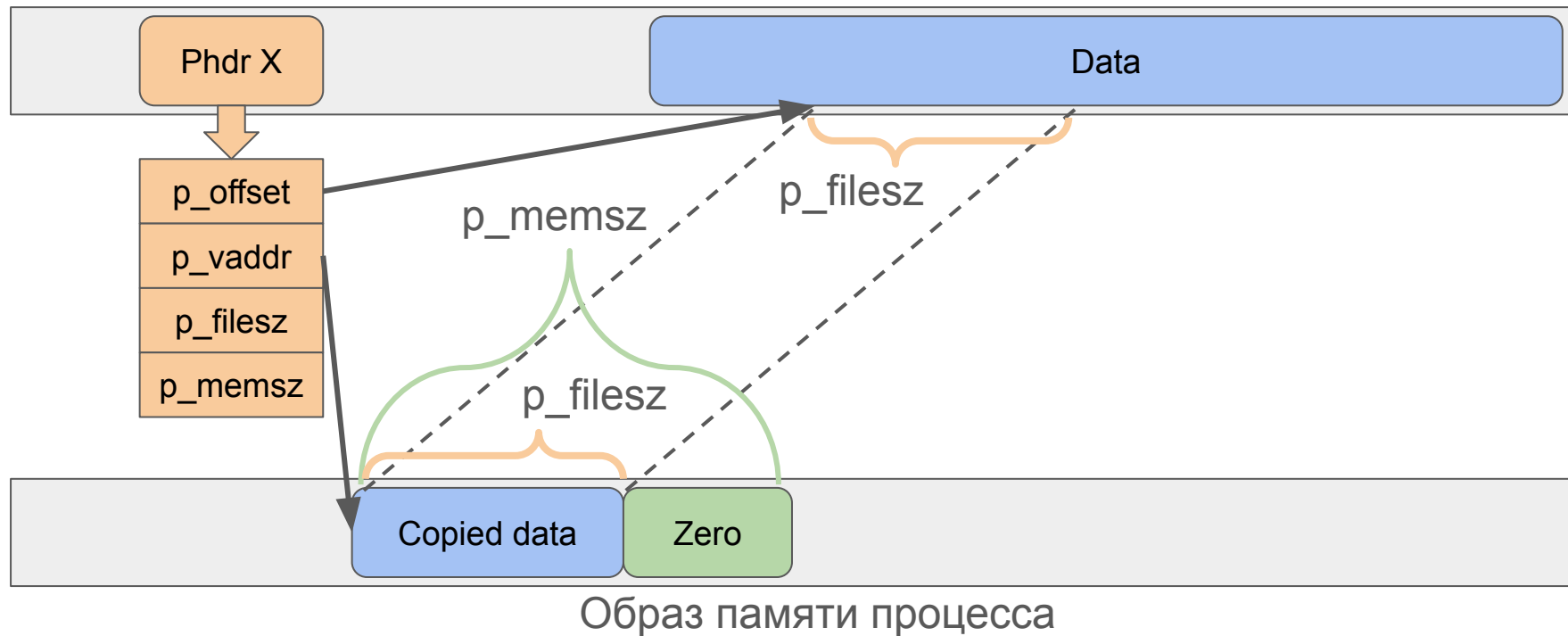
$2^N - 1$



Загрузка ELF

Загрузка ELF

- Загружаем сегменты с PT_LOAD



Загрузка ELF. Библиотеки

- Мы знаем:
 - Как устроен ELF
 - Какие данные и куда нужно загружать из файла
- Будем читать структуру заголовка из файла, искать таблицу заголовков сегментов по смещению
- Читаем таблицу заголовков сегментов
- Ищем нужные сегменты...
- А может есть библиотека для этого?

Загрузка ELF. Библиотеки

- Elfio - <https://github.com/serge1/ELFIO>
- ELFIO::elfio – класс для работы с ELF файлом
 - load(filename) – загрузка существующего файла
 - get_class() – возвращает тип файла (ELFCLASS32, ELFCLASS64)
 - get_encoding() – порядок байтов (ELFDATA2LSB, ELFDATA2MSB)
 - segments – массив сегментов
- ELFIO::segment – класс для работы с сегментами
 - get_type() - Возвращает тип сегмента
 - get_virtual_address() - виртуальный адрес сегмента
 - get_file_size() - размер сегмента в файле
 - get_memory_size() - размер сегмента в памяти
 - get_data() - указатель на начало сегмента

Elfio hello world

```
int main(int argc, char* argv[]) {
    using namespace ELFIO;
    elfio reader; // Create elfio reader
    // Print ELF file segments info
    Elf_Half seg_num = reader.segments.size();
    std::cout << "Number of segments: " << seg_num << std::endl;
    for ( int i = 0; i < seg_num; ++i ) {
        const segment* pseg = reader.segments[i];
        std::cout << " [" << i << "] 0x" << std::hex
        << pseg->get_flags() << "\t0x"
        << pseg->get_virtual_address() << "\t0x"
        << pseg->get_file_size() << "\t0x"
        << pseg->get_memory_size() << std::endl;

        const char* p = reader.segments[i]->get_data(); // Access segments's data
    }
}
```

ELF utilities

ELF utilities

- А как быстро посмотреть в elf?
- Существует как минимум 2 инструмента
 - readelf
 - objdump

readelf

- Простая читалка формата
- **Не зависит от архитектуры**, под которую скомпилирован elf
- Читает формат в общем виде

```
> readelf -h demo.elf
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:             ELF64
  Data:              2's complement, little endian
  Version:           1 (current)
  OS/ABI:             UNIX - System V
  ABI Version:       0
  Type:              DYN (Position-Independent Executable file)
  Machine:           Advanced Micro Devices x86-64
  Version:           0x1
  Entry point address: 0x1040
  Start of program headers: 64 (bytes into file)
  Start of section headers: 13920 (bytes into file)
  Flags:              0x0
  Size of this header: 64 (bytes)
  Size of program headers: 56 (bytes)
  Number of program headers: 13
  Size of section headers: 64 (bytes)
  Number of section headers: 29
  Section header string table index: 28
```

???

```
> readelf -S demo.elf
There are 29 section headers, starting at offset 0x3660:

Section Headers:
 [Nr] Name              Type              Address             Offset
     Size              EntSize          Flags  Link  Info  Align
-----
 [ 0]                     NULL              0000000000000000    00000000
     0000000000000000  0000000000000000          0   0   0
[14] .text                PROGBITS          0000000000001040    00001040
     00000000000000f8  0000000000000000    AX      0   0   16
```

```
> readelf -l demo.elf

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1040
There are 13 program headers, starting at offset 64

Program Headers:
 Type           Offset             VirtAddr           PhysAddr
   FileSiz      MemSiz          Flags             Align
-----
 PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
                0x0000000000002d8 0x0000000000002d8  R               0x8
 INTERP         0x0000000000000318 0x0000000000000318 0x0000000000000318
                0x000000000000001c 0x000000000000001c  R               0x1
 [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
 LOAD           0x00000000000005f0 0x00000000000005f0 0x00000000000005f0
                0x00000000000005f0 0x00000000000005f0  R               0x1000
 LOAD           0x0000000000001000 0x0000000000001000 0x0000000000001000
                0x0000000000000145 0x0000000000000145  R E             0x1000
 LOAD           0x0000000000002000 0x0000000000002000 0x0000000000002000
                0x00000000000000c4 0x00000000000000c4  R               0x1000
 LOAD           0x0000000000002df0 0x0000000000002df0 0x0000000000002df0
                0x0000000000000220 0x0000000000000228  RW              0x1000
```

objdump

- Архитектурно-зависимый инструмент
- Может дизассемблировать содержимое ELF
- Является частью тулчейна
 - riscv64-unknown-elf-objdump

```
objdump -d demo.elf
```



```
> objdump -d riscv.elf
```

```
riscv.elf:      file format elf32-little
```

```
objdump: can't disassemble for architecture UNKNOWN!
```

```
> readelf -h riscv.elf
```

```
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                           ELF32
  Data:                               2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                               EXEC (Executable file)
  Machine:                           RISC-V
```

```
00000000000001129 <main>:
   1129:      f3 0f 1e fa                endbr64
   112d:      55                        push    %rbp
   112e:      48 89 e5                mov     %rsp,%rbp
   1131:      b8 00 00 00 00          mov     $0x0,%eax
   1136:      5d                        pop     %rbp
   1137:      c3                        ret
```